

ShopCo

Security Remediation Reference

All Eight Fixes · Why They Work · Cross-Language Equivalents · MITRE CWE / CAPEC / ATT&CK

BBC Academy / JBI Training · Secure Web Application Development · 19–20 March 2026

Section A — Authentication & Access Control Remediations

- Aligned with Day 1 Module 5 OWASP A01 Broken Access Control & A07 ID & Authentication Failures

Flaws 5, 6, and 7 all arise from the same underlying design failure: the application confused identification with authorisation, and trusted caller-supplied data instead of server-authoritative data.

Flaw 5
Fix 5

Store `user_id` in session, not `username`

A07 ID & Auth
Failures

The session is the server's memory of who you are. Storing a mutable value (`username`) instead of an immutable identifier (`user_id`) means the session can be manipulated to change your effective identity without re-authenticating.

✗ VULNERABLE

```
# FLAW: stores the mutable username
# An attacker who forges the session cookie
# can set any username, including "admin"

session["user"] = row["username"]
session["role"] = row["role"]

# Later — role check trusts session:
if session.get("role") == "admin":
    # attacker sets role=admin in cookie
```

vs

✓ FIXED

```
# FIX: store the immutable integer user_id
# The ID cannot be changed by renaming an
# account or forging a cookie field.

session.clear() # prevent fixation
session["user_id"] = row["id"]

# Later — role re-fetched from DB:
user = get_current_user() # DB lookup
if user["role"] == "admin": # DB truth
```

⚙ Why this fix works:

The `user_id` is an integer assigned by the database. It cannot be changed by renaming an account, by social engineering a password reset, or by forging the session cookie. The role is re-fetched from the database on every request via `get_current_user()` — it is never stored in the session and therefore cannot be tampered with. Even if an attacker forges the session cookie (because they discovered the secret key), they can only supply an integer; the role check still goes to the database.

⚡ The Invariant Principle:

Never store in a session any value that can change independently of the user's identity, or any value whose integrity you cannot verify without a database lookup.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / Flask	<code>session["user_id"] = user.id # integer, not username</code>
Java / Spring	<code>HttpSession.setAttribute("userId", user.getId()); // Long, not String</code>
Node / Express	<code>req.session.userId = user.id; // number, never req.session.username</code>
C# / ASP.NET	<code>HttpContext.Session.SetInt32("UserId", user.Id);</code>
PHP	<code>\$_SESSION['user_id'] = (int)\$user['id']; // cast to int explicitly</code>
Go / Gorilla	<code>session.Values["user_id"] = user.ID // typed int, not interface{}</code>

MITRE References			
CWE	CWE-302	Authentication Bypass by Assumed-Immutable Data	https://cwe.mitre.org/data/definitions/302.html
CWE	CWE-384	Session Fixation	https://cwe.mitre.org/data/definitions/384.html
CAPEC	CAPEC-196	Session Credential Falsification through Forging	https://capec.mitre.org/data/definitions/196.html
ATT&CK	T1550.004	Use Alternate Authentication Material: Web Session Cookie	https://attack.mitre.org/techniques/T1550/004/

Spot it in any codebase — look for:

- `session["username"] = ...` or `session["user"] = username_variable`
- `if session.get("role") == "admin"` — role stored in session, not from DB
- `req.session.user = user` — entire user object serialised into session
- Cookie contains a username or email address in plain text or base64

Flaw 6 Fix 6	Add authentication decorator to every protected route	A01 Broken Access Control
-----------------	---	---------------------------

A route with no authentication check is accessible to any HTTP client in the world. The browser navigation bar is not a security control — anyone can type a URL directly.

✗ VULNERABLE	vs	✓ FIXED
<pre># FLAW: route has no auth check # Any unauthenticated visitor can # call GET /profile @app.route("/profile") def profile(): user_id = request.args.get("user_id")</pre>		<pre># FIX: @login_required enforces auth # Unauthenticated requests are # redirected to /login @app.route("/profile") @login_required # ← added def profile():</pre>

```
# ... returns user data
```

```
user = get_current_user()
```

```
# The decorator itself – reusable across every protected route:
import functools

def login_required(f):
    @functools.wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            return redirect(url_for("login", next=request.path))
        return f(*args, **kwargs)
    return decorated
```

⚙️ Why this fix works:

The decorator wraps every protected route in a single, centrally maintained authentication check. Adding it to a new route takes one line. Removing it from a route requires a deliberate action — accidental omission is now the obvious deviation from the standard pattern. Centralisation also means that if the authentication mechanism changes (e.g. switching from session to JWT), you update one function, not every route.

⚡ The Invariant Principle:

Authentication checks must be applied at the route boundary by a mechanism that is on by default or requires explicit opt-in, not explicit opt-out.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / Flask	@login_required (custom decorator) or flask-login: @flask_login.login_required
Python / FastAPI	Depends(get_current_user) injected into route signature
Java / Spring	@PreAuthorize("isAuthenticated()") or Spring Security filter chain
Node / Express	router.use(authMiddleware) applied to entire router, not per-route
C# / ASP.NET	[Authorize] attribute on controller or action – default-deny at controller level
PHP / Laravel	Route::middleware(["auth"])->group(...) or \$this->middleware("auth")
Go / Gin	authorized := r.Group("/") { authorized.Use(AuthRequired()) }

🔗 MITRE References

CWE	CWE-306	Missing Authentication for Critical Function	https://cwe.mitre.org/data/definitions/306.html
CWE	CWE-862	Missing Authorization	https://cwe.mitre.org/data/definitions/862.html
CAPEC	CAPEC-87	Forceful Browsing	https://capec.mitre.org/data/definitions/87.html
ATT&CK	T1190	Exploit Public-Facing Application	https://attack.mitre.org/techniques/T1190/

🔍 Spot it in any codebase — look for:

- @app.route(...) with NO @login_required or @jwt_required on the next line

- Route handler that checks session only when a specific condition is met (conditional auth)
- A test that hits an endpoint without an Authorization header and receives 200 OK
- Route list shows admin/internal routes without a middleware group applied to them

Flaw 7
Fix 7

Derive user identity from session, never from URL parameters

A01 Broken Access Control / IDOR

Insecure Direct Object Reference (IDOR): allowing a caller to supply the key used to retrieve their own (or another user's) record. The fix is to make the server determine the subject from its own trusted state, never from the request.

✗ VULNERABLE

```
# FLAW: user_id comes from the URL
# ?user_id=1 → profile of user 1
# ?user_id=2 → profile of user 2
# No authentication required (Flaw 6)

@app.route("/profile")
def profile():
    user_id = request.args.get("user_id", "1")
    row = db.execute(
        f"SELECT * FROM users WHERE id={user_id}"
    ).fetchone()
```

vs

✓ FIXED

```
# FIX: user_id comes from the server-side
# session — the caller cannot supply it.
# A user can ONLY see their own profile.

@app.route("/profile")
@login_required
def profile():
    user = get_current_user() # session only
    # No URL parameter accepted at all
```

⚙ Why this fix works:

The `user_id` is sourced exclusively from `session["user_id"]`, which was set by the server at login and is bound to the authenticated session. The route accepts no `user_id` URL parameter — the parameter does not exist. A legitimate user cannot (and has no reason to) specify whose profile to view: they always get their own. An admin viewing another user's profile is a separate, explicitly authorised admin route with its own role check.

This is a design fix, not a coding fix. The IDOR flaw existed because the design never asked: "Who should be able to call this endpoint and for which records?" Threat modelling would have surfaced this at design time.

⚡ The Invariant Principle:

The subject of any access-controlled operation must be derived from the server's trusted state, never from a caller-supplied identifier — unless explicit authorisation exists to access other subjects.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / Flask	<code>user_id = session["user_id"] # never request.args.get("user_id")</code>
Java / Spring	<pre>Long userId = ((UserDetails)SecurityContextHolder.getContext().getAuthentication().getPrincipal()).getId();</pre>

Node / Express	<code>const userId = req.user.id; // from passport.js / JWT middleware, not req.params.id</code>
C# / ASP.NET	<code>var userId = User.FindFirstValue(ClaimTypes.NameIdentifier); // from ClaimsPrincipal</code>
PHP	<code>\$userId = (int)\$_SESSION['user_id']; // never \$_GET['user_id']</code>
Go	<code>userId := ctx.Value(authKey).(int) // from auth middleware context, not URL param</code>

🔗 MITRE References			
CWE	CWE-639	Authorization Bypass Through User-Controlled Key	https://cwe.mitre.org/data/definitions/639.html
CWE	CWE-284	Improper Access Control	https://cwe.mitre.org/data/definitions/284.html
CAPEC	CAPEC-122	Privilege Abuse	https://capec.mitre.org/data/definitions/122.html
CAPEC	CAPEC-1	Accessing/Intercepting/Modifying HTTP Communication	https://capec.mitre.org/data/definitions/1.html
ATT&CK	T1078	Valid Accounts (lateral data access)	https://attack.mitre.org/techniques/T1078/

🔍 Spot it in any codebase — look for:

- `request.args.get("user_id")` or `req.params.userId` or `$_GET["id"]` used to look up user data
- `GET /api/users/:id` — if any authenticated user can request any `:id` value
- `for-loop` or `bulk fetch` where the caller controls the ID array
- URL contains `/profile/123` or `/account/456` — test: change the integer, see if you get another user's data

Section B — Security Misconfiguration Remediation

- Aligned with Day 2 Module 8 A02 Security Misconfiguration & A03 Software Supply Chain Failures

Fix 2 is a configuration failure: a development-only setting reached production. Unlike code bugs, misconfiguration is often invisible in the source code — it requires a separate configuration review step in the SDLC.

Flaw 2 Fix 2	Disable DEBUG mode by default; gate it on an environment variable	A02 Security Misconfiguration
-------------------------------	--	--------------------------------------

Werkzeug's interactive debugger (enabled when DEBUG=True) provides an in-browser Python REPL attached to the server process. Any visitor who triggers an unhandled exception can execute arbitrary Python. This is remote code execution with zero authentication.

✗ VULNERABLE	vs	✓ FIXED
<pre># FLAW: hardcoded True in source code # reaches production with every deploy app.config["DEBUG"] = True # Werkzeug REPL available to anyone # who triggers a 500 error: # → Full stack trace in browser # → "Click here to open console" # → Execute Python as server user</pre>		<pre># FIX: read from environment variable # Production: FLASK_DEBUG is unset → False # Dev only: export FLASK_DEBUG=true app.config["DEBUG"] = os.environ.get("FLASK_DEBUG", "false").lower() == "true" # Config/secrets review catches any # deployment where FLASK_DEBUG=true</pre>

⚙️ Why this fix works:

The fix separates the configuration from the code. The source file is now safe to deploy to any environment — there is no hardcoded dangerous value that could accidentally reach production. The presence of FLASK_DEBUG=true in a production environment is now a discoverable, auditable deployment error rather than a code defect buried in a repository.

Additionally: custom error handlers (403, 404, 500 in vulnerable_shop_fixed.py) replace Werkzeug's default error pages with generic messages, even if DEBUG were somehow enabled. The full exception is logged server-side only.

⚡ The Invariant Principle:

Development-only features must be gated on environment variables that default to OFF, never on hardcoded values that default to ON.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / Flask	<code>app.config["DEBUG"] = os.environ.get("FLASK_DEBUG", "false").lower()=="true"</code>
Python / Django	<code>DEBUG = env.bool("DJANGO_DEBUG", default=False) # using django-envron</code>
Java / Spring	<code>spring.profiles.active=prod → application-prod.properties has debug=false</code>
Node / Express	<code>app.set("env", process.env.NODE_ENV "production") — never hardcode "development"</code>

C# / ASP.NET	ASPNETCORE_ENVIRONMENT=Production in deployment; never Development in server env vars
PHP / Laravel	APP_DEBUG=false in .env.production — never commit .env with APP_DEBUG=true
Go / Gin	gin.SetMode(gin.ReleaseMode) — set before any router initialisation; gate on env var

🔗 MITRE References			
CWE	CWE-489	Active Debug Code	https://cwe.mitre.org/data/definitions/489.html
CWE	CWE-215	Insertion of Sensitive Information Into Debugging Code	https://cwe.mitre.org/data/definitions/215.html
CWE	CWE-94	Improper Control of Generation of Code (Debug RCE)	https://cwe.mitre.org/data/definitions/94.html
CAPEC	CAPEC-121	Exploit Non-Production Interfaces	https://capec.mitre.org/data/definitions/121.html
ATT&CK	T1190	Exploit Public-Facing Application (via debug endpoint)	https://attack.mitre.org/techniques/T1190/
ATT&CK	T1082	System Information Discovery (via stack traces)	https://attack.mitre.org/techniques/T1082/

🔍 Spot it in any codebase — look for:

- DEBUG = True or debug=True hardcoded in any source file that is deployed
- stack traces in HTTP responses visible in ZAP or browser (not just during development)
- Werkzeug debugger PIN challenge page visible to unauthenticated users
- Server header or X-Powered-By header reveals framework name and version
- Error response body contains class names, file paths, or SQL query text

Section C — Cryptographic Failures Remediations

- Aligned with Day 2 Module 9 OWASP A04 Cryptographic Failures & A08 Software & Data Integrity Failures

Fixes 1 and 3 both represent failures of the same kind: using a cryptographic primitive for its speed rather than its security. MD5 is fast — which is exactly why it is wrong for passwords. A hardcoded secret key is convenient — which is exactly why it is wrong for session signing.

Flaw 1	Load secret key from environment variable; never	A04 Cryptographic Failures
Fix 1	hardcode it	

Flask's `secret_key` is used to cryptographically sign session cookies. An attacker who knows the key can forge any session cookie — setting arbitrary `user_id`, `role`, or any other session data — without ever logging in.

✗ VULNERABLE	vs	✓ FIXED
<pre># FLAW: trivial hardcoded string # Committed to source control. # Anyone with repo access can forge # every session cookie ever issued. app.secret_key = "password" # flask-unsign --sign --secret password # --cookie {"user_id":1,"role":"admin"} # → valid admin session in 2 seconds</pre>		<pre># FIX: load from environment variable # Never appears in source or git history. # Generate: # export FLASK_SECRET_KEY=\$(openssl rand # -hex 32) app.secret_key = os.environ.get("FLASK_SECRET_KEY", os.urandom(32) # dev-only fallback) # os.urandom(32) = 256 bits of entropy</pre>

⚙️ Why this fix works:

`os.environ.get("FLASK_SECRET_KEY")` reads the key from the process environment, which is set at deployment time and is never committed to source control. A 32-byte (256-bit) random key generated by `openssl rand -hex 32` has 2^{256} possible values — computationally infeasible to brute-force.

The `os.urandom(32)` fallback changes on every process restart, making it suitable for development (where session persistence does not matter) but not for production (where it would invalidate all sessions on every restart, prompting a developer to set the env var correctly).

Critically: environment variables can be managed by secrets management systems (AWS Secrets Manager, HashiCorp Vault, GCP Secret Manager) and rotated without touching source code.

⚡ The Invariant Principle:

Cryptographic secrets must not appear in source code, configuration files, or version control. They belong in a secrets management system or environment variables set at deployment time.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / Flask	<code>app.secret_key = os.environ["FLASK_SECRET_KEY"] # KeyError if unset = fail-fast</code>
Python / Django	<code>SECRET_KEY = env("DJANGO_SECRET_KEY") # django-environ raises if missing</code>

Java / Spring	@Value("\${app.jwt.secret}") from application.properties; value in Vault/env
Node / Express	secret: process.env.SESSION_SECRET - npm dotenv for local dev, never commit .env
C# / ASP.NET	builder.Configuration["JwtSettings:Secret"] from appsettings.json + Azure Key Vault override
PHP	\$secret = \$_ENV["APP_KEY"]; - use phpdotenv; never hardcode in config.php
Go	os.Getenv("JWT_SECRET") + if secret == "" { log.Fatal("JWT_SECRET not set") }

MITRE References			
CWE	CWE-321	Use of Hard-coded Cryptographic Key	https://cwe.mitre.org/data/definitions/321.html
CWE	CWE-798	Use of Hard-coded Credentials	https://cwe.mitre.org/data/definitions/798.html
CAPEC	CAPEC-191	Read Sensitive Constants Within an Executable	https://capec.mitre.org/data/definitions/191.html
ATT&CK	T1552.001	Unsecured Credentials: Credentials In Files	https://attack.mitre.org/techniques/T1552/001/
ATT&CK	T1552.004	Unsecured Credentials: Private Keys	https://attack.mitre.org/techniques/T1552/004/

Spot it in any codebase — look for:

- secret_key = "..." or SECRET_KEY = "..." any literal string assigned directly
- git log --all -S "secret" — search git history for secrets (they persist after deletion)
- grep -r "password|secret|apikey|token" --include="*.py" src/
- truffleHog or gitleaks run on the repository — standard CI/CD gate
- flask-unsign --decode --cookie [session_cookie] — check if the session is decodable

Flaw 3	Use bcrypt (or Argon2) instead of MD5 for password hashing	A04 Cryptographic Failures
Fix 3		

MD5 was designed to be fast. Modern GPUs compute 60 billion MD5 hashes per second. A database of MD5-hashed passwords is crackable in hours. bcrypt is designed to be slow — its cost factor directly controls attacker throughput.

✗ VULNERABLE		✓ FIXED
<pre># FLAW: MD5 — fast, unsalted, broken # GPU: ~60,000,000,000 hashes/sec # "letmein" → cracked in milliseconds # from any rockyou.txt wordlist pw_hash = hashlib.md5(password.encode()).hexdigest() # Verify: if pw_hash == row["password_hash"]:</pre>	vs	<pre># FIX: bcrypt — slow, salted, secure # rounds=12 → ~250ms per hash # GPU: ~120,000 hashes/sec (500,000x slower) # Salt included in stored hash automatically pw_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12)).decode() # Verify (constant-time): if bcrypt.checkpw(pwd.encode(), stored):</pre>

Why this fix works:

bcrypt applies a configurable work factor (rounds). With rounds=12, each hash takes approximately 250ms on typical server hardware. An attacker who steals your database can attempt ~120,000 bcrypt hashes per second on a GPU (vs 60 billion for MD5). A 100-million-entry wordlist that would crack MD5 hashes in 2 seconds would take 10 days against bcrypt rounds=12.

bcrypt automatically generates and stores a unique random salt per password, eliminating rainbow table attacks entirely. Each password's stored hash begins with \$2b\$12\$[22-char-salt][31-char-hash].

bcrypt.checkpw() uses a constant-time comparison that prevents timing attacks — an attacker cannot determine whether a hash is "close" to correct by measuring response time.

⚡ The Invariant Principle:

Password hashing must use a slow, salted, adaptive algorithm (bcrypt, scrypt, or Argon2). Never use MD5, SHA-1, SHA-256, or any general-purpose hashing function, regardless of salt.

Performance comparison — why slowness is the security property

Algorithm	GPU hashes/sec	Time to crack rockyou.txt (14M)	Verdict
MD5	~60,000,000,000	< 1 second	✗ Never use
SHA-1	~22,000,000,000	< 1 second	✗ Never use
SHA-256	~11,000,000,000	~1 second	✗ Never use
bcrypt rounds=10	~150,000	~26 hours	✓ Acceptable
bcrypt rounds=12	~37,000	~4.4 days	✓ Recommended
Argon2id	~10,000	~16 days	✓ Best practice

Cross-language equivalents

Language	Safe equivalent / pattern
Python	<code>import bcrypt → bcrypt.hashpw(pwd.encode(), bcrypt.gensalt(rounds=12))</code>
Python (alt)	<code>from passlib.hash import argon2 → argon2.hash(password) (Argon2id)</code>
Java / Spring	<code>new BCryptPasswordEncoder(12).encode(rawPassword) (Spring Security built-in)</code>
Node.js	<code>bcrypt.hash(password, 12) / argon2.hash(password, {type: argon2.argon2id})</code>
C# / ASP.NET	<code>BCrypt.Net.BCrypt.HashPassword(password, workFactor: 12) or ASP.NET Identity default</code>
PHP	<code>password_hash(\$pwd, PASSWORD_BCRYPT, ["cost"=>12]) + password_verify(\$pwd, \$hash)</code>
Go	<code>bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost) — golang.org/x/crypto/bcrypt</code>

MITRE References

CWE	CWE-327	Use of a Broken or Risky Cryptographic Algorithm	https://cwe.mitre.org/data/definitions/327.html
CWE	CWE-916	Use of Password Hash With Insufficient Computational Effort	https://cwe.mitre.org/data/definitions/916.html
CAPEC	CAPEC-55	Rainbow Table Password Cracking	https://capec.mitre.org/data/definitions/55.html
CAPEC	CAPEC-49	Password Brute Forcing	https://capec.mitre.org/data/definitions/49.html
ATT&CK	T1110.002	Brute Force: Password Cracking	https://attack.mitre.org/techniques/T1110/002/

🔍 Spot it in any codebase — look for:

- import hashlib followed by .md5(), .sha1(), or .sha256() in any auth or password context
- password_hash() in PHP without PASSWORD_BCRYPT or PASSWORD_ARGON2ID as the second argument
- MessageDigest.getInstance("MD5") or "SHA-1" or "SHA-256" in Java password-handling code
- Hash output is a 32-char hex string (MD5) or 40-char hex string (SHA-1) — bcrypt output starts with \$2b\$
- No salt column in user table — unsalted hashes are trivially rainbow-table crackable

Section D — Injection Remediations

- Aligned with Day 1, Module 06 A05 Injection

Fixes 4 and 8 address the same root cause from different angles. In both cases, user-controlled data was allowed to cross a boundary and be interpreted as code. The structural fix is identical in principle: separate the data channel from the code channel, then treat the data channel as untrusted by default.

Flaw 4 Fix 4	Replace f-string SQL with parameterised queries throughout	A05 Injection (SQL Injection)
-------------------------------	---	--------------------------------------

SQL injection occurs when user input is concatenated into a SQL string. The database parser cannot distinguish between the developer's SQL structure and the attacker's injected SQL. Parameterised queries use a different communication channel for data — it physically cannot alter the query structure.

✗ VULNERABLE	vs	✓ FIXED
<pre># FLAW (3 instances in ShopCo) # Login — authentication bypass: query = (f"SELECT * FROM users WHERE" f" username='{username}'" f" AND pw_hash='{pw_hash}''") # Search — UNION data extraction: query = (f"SELECT * FROM products" f" WHERE name LIKE '%{q}%'") # Profile — also injectable: f"SELECT * FROM users WHERE id={user_id}"</pre>		<pre># FIX: parameterised queries everywhere # Login — SQL structure is fixed: row = db.execute("SELECT * FROM users WHERE" " username = ?" " AND password_hash = ?", (username, pw_hash) # data channel).fetchone() # Search: rows = db.execute("SELECT * FROM products" " WHERE name LIKE ?", (f"%{q}%",) # data channel).fetchall()</pre>

⚙️ Why this fix works:

A parameterised query sends the SQL structure and the data values to the database in separate operations. The database driver transmits the SQL text first (with ? placeholders), then sends the parameter values. The parser processes the SQL before it ever sees the data. There is no string concatenation — there is no SQL string that contains user input.

The critical detail: the LIKE query uses (f"%{q}%",) as the parameter value. The % wildcard characters are part of the parameter, not the query structure. If a user searches for %' UNION SELECT... the entire string — including the percent signs, the quote, and the keyword UNION — is treated as a literal search term, not SQL.

Parameterised queries are not the same as string escaping. Escaping can be bypassed in certain character encodings and requires every developer to remember to do it every time. Parameterised queries make injection structurally impossible regardless of the input value.

⚡ The Invariant Principle:

SQL query structure and user-supplied values must travel to the database via separate channels. Concatenation is the vulnerability. Parameterisation is the only structural fix.

Cross-language equivalents

Language	Safe equivalent / pattern
Python / SQLite	<code>cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))</code>
Python / SQLAlchemy	<code>db.execute(text("SELECT * FROM users WHERE id = :id"), {"id": user_id})</code>
Python / SQLAlchemy ORM	<code>User.query.filter_by(username=username).first()</code> — ORM parameterises automatically
Java / JDBC	<code>PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE id = ?"); ps.setInt(1, userId);</code>
Java / Spring JPA	<code>@Query("SELECT u FROM User u WHERE u.id = :id") User findById(@Param("id") Long id);</code>
Node / mysql2	<code>conn.query("SELECT * FROM users WHERE id = ?", [userId], callback)</code>
Node / Sequelize	<code>User.findOne({ where: { username: username } })</code> — ORM handles parameterisation
C# / ADO.NET	<code>cmd.CommandText = "SELECT * FROM Users WHERE Id = @Id"; cmd.Parameters.AddWithValue("@Id", userId);</code>
PHP / PDO	<code>\$stmt = \$pdo->prepare("SELECT * FROM users WHERE id = ?"); \$stmt->execute([\$userId]);</code>
Go / database/sql	<code>db.QueryRow("SELECT * FROM users WHERE id = \$1", userId)</code>

🔗 MITRE References

CWE	CWE-89	Improper Neutralization of Special Elements in SQL Commands	https://cwe.mitre.org/data/definitions/89.html
CWE	CWE-943	Improper Neutralization of Special Elements in Data Query Logic	https://cwe.mitre.org/data/definitions/943.html
CAPEC	CAPEC-66	SQL Injection	https://capec.mitre.org/data/definitions/66.html
CAPEC	CAPEC-7	Blind SQL Injection	https://capec.mitre.org/data/definitions/7.html
ATT&CK	T1190	Exploit Public-Facing Application	https://attack.mitre.org/techniques/T1190/
ATT&CK	T1078	Valid Accounts (credential theft via SQLi)	https://attack.mitre.org/techniques/T1078/

🔍 Spot it in any codebase — look for:

- `f"SELECT ... WHERE {variable}"` or `"SELECT ... WHERE " + variable` — any string concatenation in SQL
- `".format(username=username)` or `% interpolation (%s % variable)` in a SQL string
- `cursor.execute(query)` where query is built before the execute call
- `ORDER BY {column}` or `LIMIT {n}` — dynamic SQL clauses are also injectable (use an allowlist for these)
- The test: enter a single quote `'` in every input field. If you see a database error, SQLi is present.

Flaw 8
Fix 8

Use subprocess with shell=False and allowlist input validation

A05 Injection
(Command Injection)

Command injection occurs when user input is passed to a shell. The shell interprets metacharacters (`;` `&&` `||` `|` `$()` ```) as command separators or substitution operators. `shell=False` bypasses the shell entirely; allowlist validation provides a second independent layer.

✗ VULNERABLE

```
# FLAW: os.popen invokes a shell
# Shell interprets metacharacters

output = os.popen(
    f"ping -c 2 {host}"
).read()

# Input: 127.0.0.1; cat /etc/passwd
# Shell executes: ping -c 2 127.0.0.1
# AND: cat /etc/passwd
# Both outputs returned to browser
```

vs

✓ FIXED

```
# FIX A: allowlist input validation
SAFE = re.compile(r"[a-zA-Z0-9.\-]{1,253}")
if not SAFE.fullmatch(host):
    return error("Invalid host")

# FIX B: shell=False — no shell spawned
result = subprocess.run(
    ["ping", "-c", "2", host],
    shell=False,      # ← key
    capture_output=True, text=True,
    timeout=10)
```

⚙️ Why each layer of this fix works:

Fix 8a — Allowlist validation: the regex `[a-zA-Z0-9.\-]{1,253}` permits only characters that appear in valid hostnames and IPv4 addresses. Shell metacharacters (semicolons, pipes, dollar signs, backticks, ampersands, spaces) match none of these characters and are rejected before any subprocess call is made. A small allowlist of safe characters is always stronger than a blocklist of dangerous ones — attackers can always find characters you forgot to block.

Fix 8b — `shell=False`: `subprocess.run([...], shell=False)` passes the command as a Python list. The operating system's `exec()` system call receives ping as the program name and the host value as a literal argument string. There is no shell to interpret the semicolon as a command separator. Even if the allowlist were somehow bypassed, `shell=False` means the injected metacharacters arrive as literal text in the argument to ping — they cannot chain a second command.

Defence in depth: use both layers. Either one alone is a significant improvement; together they are structurally sound.

⚡ The Invariant Principle:

Never pass user input to a shell. Pass the command and arguments as a list to the OS directly. When the task requires a shell, allowlist the input to only characters that are valid in that context.

Shell metacharacters — the complete injection character set

```

; - command separator      && - AND then      || - OR then
| - pipe to next command   $() - command subst.  `` - command subst.
> - redirect output        >> - append output  < - redirect input
& - background process     \n - newline (shell)  \r - carriage return

```

Rule: if any of these can reach a shell, command injection is possible.
 Rule: if shell=False, none of these characters have special meaning.

Cross-language equivalents

Language	Safe equivalent / pattern
Python	<code>subprocess.run(["ping", "-c", "2", host], shell=False)</code> — never <code>os.popen()</code> or <code>os.system()</code>
Java	<code>ProcessBuilder pb = new ProcessBuilder("ping", "-c", "2", host);</code> // list constructor, no shell
Node.js	<code>child_process.execFile("ping", ["-c", "2", host])</code> — never <code>exec()</code> with template literal
C# / .NET	<code>Process.Start(new ProcessStartInfo { FileName="ping", ArgumentList={"-c", "2", host} })</code>
PHP	<code>escapeshellarg(\$host) + pcntl_exec()</code> — but best practice: avoid shell execution entirely in PHP
Go	<code>exec.Command("ping", "-c", "2", host)</code> — variadic args, no shell, safe by default
Ruby	<code>system("ping", "-c", "2", host)</code> — array form, no shell vs <code>`ping -c 2 #{host}`</code> — NEVER

MITRE References

CWE	CWE-78	Improper Neutralization of Special Elements in OS Commands	https://cwe.mitre.org/data/definitions/78.html
CWE	CWE-88	Improper Neutralization of Argument Delimiters in a Command	https://cwe.mitre.org/data/definitions/88.html
CWE	CWE-116	Improper Encoding or Escaping of Output	https://cwe.mitre.org/data/definitions/116.html
CAPEC	CAPEC-88	OS Command Injection	https://capec.mitre.org/data/definitions/88.html
CAPEC	CAPEC-40	Manipulating Writeable Terminal Devices	https://capec.mitre.org/data/definitions/40.html
ATT&CK	T1059	Command and Scripting Interpreter	https://attack.mitre.org/techniques/T1059/
ATT&CK	T1190	Exploit Public-Facing Application	https://attack.mitre.org/techniques/T1190/

Spot it in any codebase — look for:

- `os.popen()`, `os.system()`, `os.popen2()` — always invokes a shell
- `subprocess.run(..., shell=True)` or `subprocess.call(cmd_string, shell=True)`
- `child_process.exec(...${userInput}...)` in Node.js — template literal in `exec` = shell
- `Runtime.exec(String)` in Java — single string `exec` invokes `/bin/sh -c`
- backtick execution in PHP: ``$cmd`` or `shell_exec($cmd)` or `exec($userInput)`
- The test: enter `;id` in every text field that might reach a system call. If `uid=` appears, command injection is present.

Appendix — All 8 Fixes: Master MITRE Cross-Reference

#	Vulnerability	Remediation	OWASP 2025	CWE	CAPEC	ATT&CK
1	Hardcoded secret key	Load from env var	A04 Crypto Failures	CWE-321, CWE-798	CAPEC-191	T1552.001
2	DEBUG=True in production	Env var gate; custom error handlers	A02 Misconfiguration	CWE-489, CWE-215	CAPEC-121	T1190, T1082
3	MD5 password hashing	bcrypt rounds=12	A04 Crypto Failures	CWE-327, CWE-916	CAPEC-55, CAPEC-49	T1110.002
4	SQLi via f-string SQL	Parameterised queries	A05 Injection	CWE-89, CWE-943	CAPEC-66, CAPEC-7	T1190, T1078
5	Username in session	Store user_id; re-fetch role	A07 Auth Failures	CWE-302, CWE-384	CAPEC-196	T1550.004
6	Missing @login_required	Centralised decorator	A01 Access Control	CWE-306, CWE-862	CAPEC-87	T1190
7	IDOR via URL user_id	Derive identity from session	A01 Access Control	CWE-639, CWE-284	CAPEC-122, CAPEC-1	T1078
8	os.popen + user input	shell=False + allowlist regex	A05 Injection	CWE-78, CWE-88	CAPEC-88	T1059, T1190

Quick-access MITRE base URLs

CWE base URL	https://cwe.mitre.org/data/definitions/[CWE-NUMBER].html
CAPEC base URL	https://capec.mitre.org/data/definitions/[CAPEC-NUMBER].html
ATT&CK base URL	https://attack.mitre.org/techniques/[TECHNIQUE-ID]/
OWASP Cheat Sheets	https://cheatsheetseries.owasp.org